

- GreyCats Analytics — AI Suite Setup Guide
 - Prerequisites
 - 1. Clone & Switch Branch
 - Backend
 - Frontend
 - 2. Environment Configuration
 - Backend .env
 - Frontend .env
 - 3. Backend Setup
 - New Database Tables Created
 - New NPM Packages
 - 4. Frontend Setup
 - 5. Verify Installation
 - Backend Health Check
 - Test AI Endpoint
 - 6. AI Provider Configuration
 - Option A: System Default (Recommended for initial setup)
 - Option B: Per-User Configuration
 - 7. Supported AI Providers
 - Text Generation
 - Image Generation
 - 8. Rate Limits
 - 9. File Storage
 - 10. Troubleshooting
 - "AI provider timed out"
 - "Image generation failed"
 - "No data in chatbot context"
 - Database schema out of sync
 - Redis connection issues

GreyCats Analytics — AI Suite Setup Guide

Prerequisites

- Node.js v18+
 - MySQL 8.0+ (running via Docker or standalone)
 - Redis (running via Docker or standalone)
 - OpenAI API key with access to **gpt-4o** and **gpt-image-1**
-

1. Clone & Switch Branch

Backend

```
git clone git@github.com:aqibchauhan/Greycats-Analysis-.git
cd Greycats-Analysis-
git checkout feature/ai-suite
```

Frontend

```
git clone git@github.com:hemantsonawanegreycats/GreyCats-Analytics.git
cd GreyCats-Analytics
git checkout feature/ai-suite
```

2. Environment Configuration

Backend **.env**

Add the following variables to your existing **.env** file in the backend root:

```
# —— AI Text Provider ——
AI_TEXT_PROVIDER=openai-compatible
AI_TEXT_MODEL=gpt-4o
AI_TEXT_BASE_URL=https://api.openai.com/v1
```

```
# — AI Image Provider —
AI_IMAGE_PROVIDER=openai
AI_IMAGE_MODEL=gpt-image-1
AI_IMAGE_BASE_URL=

# — API Keys —
OPENAI_API_KEY=sk-proj-XXXXXXXXXXXXXXXXXXXXX
OPENAI_ORG_ID=org-XXXXXXXXXXXXXXXXXXXXX

# — Existing (must already be set) —
APP_ENCRYPTION_KEY=<your-encryption-key>
DATABASE_URL=mysql://root:root@localhost:3306/gc_analytics_uat_db
REDIS_URL=redis://localhost:6379
JWT_SECRET=<your-jwt-secret>
```

Frontend **.env**

No additional AI-specific variables needed. Ensure the following exists:

```
VITE_API_BASE_URL=http://localhost:5000/api
```

3. Backend Setup

```
cd Greycats-Analysis-

# Install dependencies (includes new: cheerio, pdf-parse, mammoth)
npm install

# Generate Prisma client
npx prisma generate

# Push schema to database (creates new AI tables)
npx prisma db push --accept-data-loss

# Start development server
npm run dev
```

New Database Tables Created

Table	Purpose
UserAIConfig	Per-user AI provider configuration
AIChatSession	Chat sessions linked to reports
AIChatMessage	Chat messages with actions and ratings
BrandProfile	Brand identity, voice, guidelines
BrandKnowledge	Uploaded brand documents with extracted text
AICalendar	AI-generated content calendars
AICalendarEntry	Individual calendar entries
AICreativeAsset	Generated captions, images, prompts

New NPM Packages

Package	Purpose
cheerio	HTML parsing for website scraping
pdf-parse	PDF text extraction for knowledge base
mammoth	DOCX text extraction for knowledge base

4. Frontend Setup

```
cd GreyCats-Analytics

# Install dependencies
npm install

# Start development server
npm run dev
```

Frontend runs on <http://localhost:5173>

5. Verify Installation

Backend Health Check

```
curl http://localhost:5000/api/health
# Expected: {"success":true,"message":"Server is healthy 🚀"}
```

Test AI Endpoint

```
# Login to get token
TOKEN=$(curl -s -X POST http://localhost:5000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"your-email","password":"your-password"}' | jq -r
  '.data.token // .token')

# Test brand scrape
curl -X POST http://localhost:5000/api/ai/brand/scrape \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"url":"https://example.com"}'
```

6. AI Provider Configuration

Option A: System Default (Recommended for initial setup)

Set the `OPENAI_API_KEY` and `OPENAI_ORG_ID` in `.env`. All users will use this key by default.

Option B: Per-User Configuration

Each user can configure their own AI provider in **Settings** → **AI Settings**:

1. Go to Settings page
2. Click "AI Settings" tab
3. Select provider (OpenAI, Anthropic, Google, Groq, etc.)
4. Enter API key

5. Save and test

Resolution order: User config → System default from `.env`

7. Supported AI Providers

Text Generation

Provider	Model Examples	Base URL
OpenAI	gpt-4o, gpt-4-turbo	https://api.openai.com/v1
Anthropic	claude-sonnet-4-6, claude-opus-4-6	(default)
Google Gemini	gemini-2.0-flash	(default)
Groq	llama-3.3-70b	https://api.groq.com/openai/v1
Mistral	mistral-large	https://api.mistral.ai/v1
Together AI	meta-llama/Meta-Llama-3-70B	https://api.together.xyz/v1
Ollama (local)	llama3, mistral	http://localhost:11434/v1

Image Generation

Provider	Model	Notes
OpenAI	gpt-image-1	Returns base64, saved locally
OpenAI	dall-e-3	Returns URL, downloaded locally

8. Rate Limits

Feature	Limit	Scope
Chat messages	100/day	Per user per report

Feature	Limit	Scope
Calendar generation	10/day	Per user
Caption generation	50/day	Per user
Content generation	30/day	Per user
Image generation	20/day	Per user

Rate limits are enforced via Redis. If Redis is unavailable, limits fail open (requests are allowed).

9. File Storage

AI-generated files are stored locally:

```
uploads/
├── ai/
│   ├── brand/           # Knowledge base uploads (PDF, DOCX, TXT, images)
│   └── creative/        # AI-generated images (PNG)
```

Ensure the `uploads/` directory is:

- Writable by the Node.js process
- Included in your backup strategy
- Served as static files (already configured in `src/index.ts`)

10. Troubleshooting

"AI provider timed out"

- AI calls have a 120-second timeout
- Check if the AI provider is reachable
- Try a simpler prompt to verify connectivity

"Image generation failed"

- Verify `OPENAI_API_KEY` has access to `gpt-image-1`
- Check that `uploads/ai/creative/` directory exists and is writable
- Check server logs for specific error messages

"No data in chatbot context"

- Ensure the report template has widgets with data
- Run `npx prisma db push --accept-data-loss` to ensure all tables are synced
- Check that the date range has actual metric data in the DB

Database schema out of sync

```
npx prisma generate  
npx prisma db push --accept-data-loss
```

Redis connection issues

- Rate limiting and cron jobs depend on Redis
- If Redis is down, AI features still work (rate limits are bypassed)
- Restart Redis: `docker restart greycats_redis`